

ARCHITECTURE & GUIDE D'IMPLÉMENTATION

**Pipeline Multi-Agents Python/Hermes pour la Création,
l'Adaptation EBEP, l'Exportation et la
Commercialisation de Séquences Pédagogiques**

TABLE DES MATIÈRES

1. Synthèse Exécutive & Architecture Globale (100% Code Native)
2. Matrice des Agents & Tokenomics (Analyse des Coûts)
3. Prompts Systèmes de Production pour les Sub-Agents
4. Scripts Python de Production (SVG, PNG, Fallback, PPTX)
5. Intégration Ops Python Natif : Google Drive & Systeme.io
6. Gestion des Risques & Mécanismes de Secours (Fallback)
7. Feuille de Route d'Implémentation Étape par Étape
8. Instructions de Conversion en Fichier PDF

1. SYNTHÈSE EXÉCUTIVE & ARCHITECTURE GLOBALE

1.1 Objectif du Système

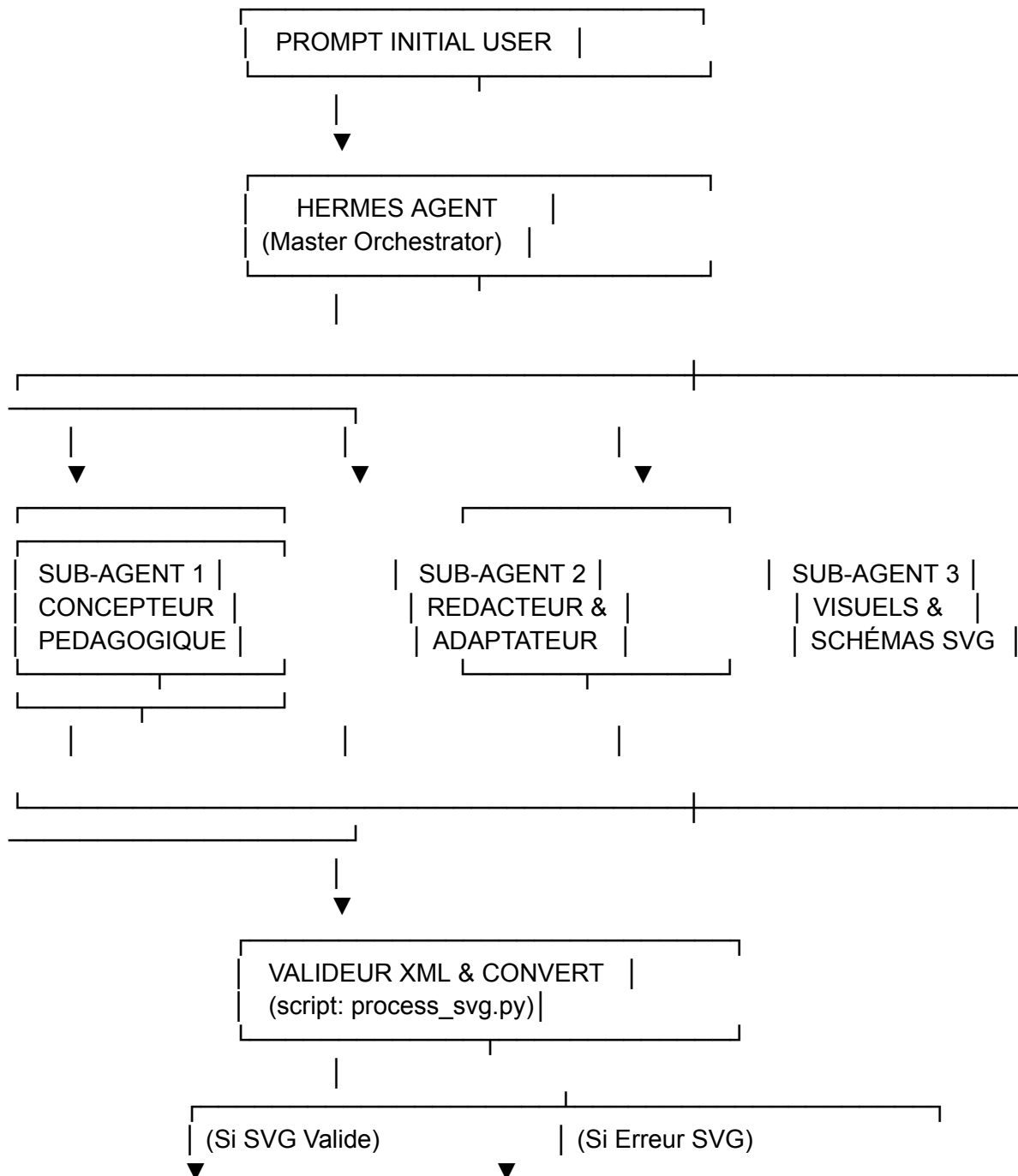
Le système automatise à 100 % la production d'une séquence pédagogique complète à partir d'une seule consigne initiale (ex: "*Séquence 4ème sur l'information et la désinformation*"), sans passer par aucun logiciel SaaS payant d'automation. Tout est exécuté en local via **Hermes Agent** et des **scripts Python**.

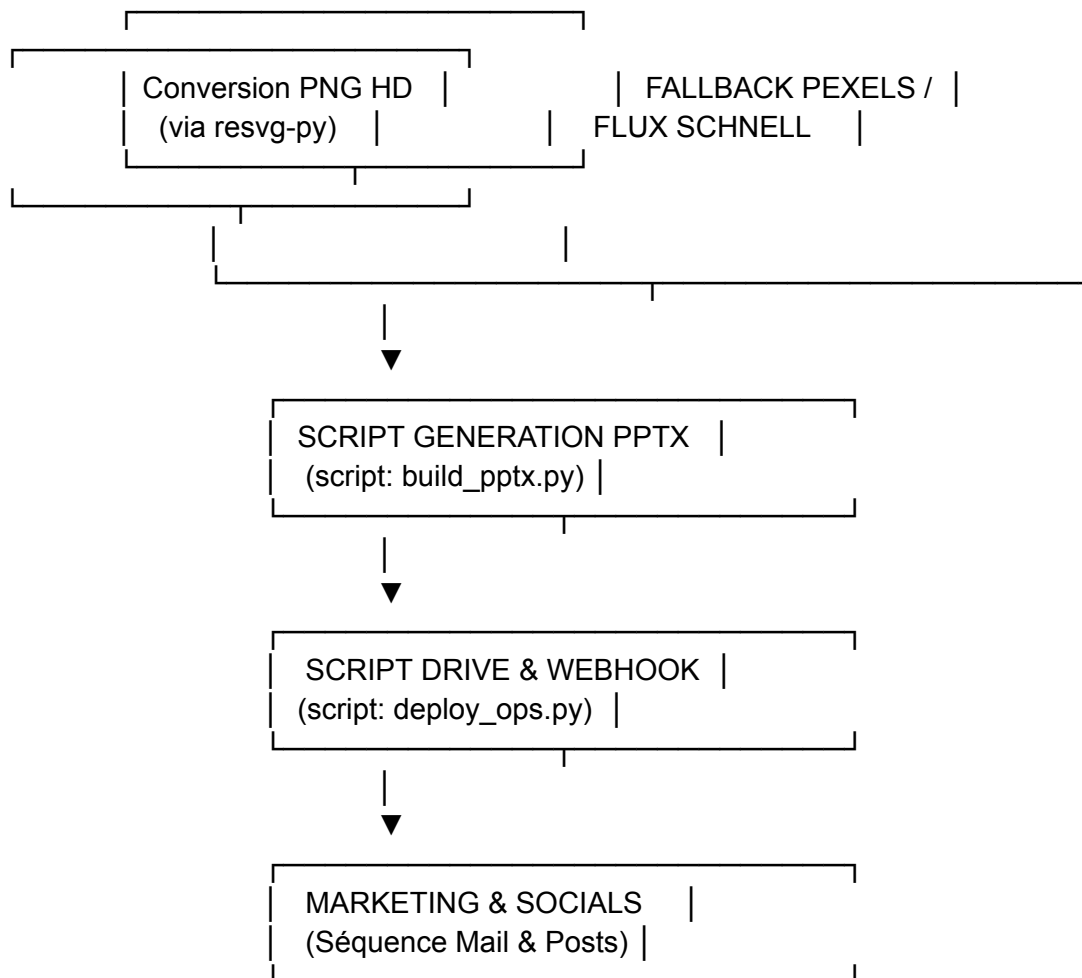
1.2 Livrables Générés par le Flux

- **Pédagogie** : Fiche Professeur, Fiche Élève, Fiche Adaptée EBEP/DYS (ergonomie renforcée).
- **Visuels** : Diaporama PowerPoint (.pptx) interactif, Schémas vectoriels SVG/PNG générés sur-mesure.

- **Multimédia** : Quizzes d'évaluation (JSON/QCM), Synthèse audio / Podcast (via NotebookLM).
- **Opérations** : Dossier structuré sur Google Drive, Fiche produit & tunnel de vente créés sur **Systeme.io**.
- **Marketing** : Séquence d'emails de vente, posts carrousels pour réseaux sociaux (LinkedIn, Instagram).

1.3 Schéma Fonctionnel du Flux (Orchestration Python & Hermes)





2. MATRICE DES AGENTS & TOKENOMICS

2.1 Configuration de la Matrice d'Agents (Mode Hybride Premium)

Agent / Composant	Technologie / Modèle	Rôle & Spécialité	Coût Exécution
Orchestrateur	Hermes Agent / Python CLI	Pilotage global, enchaînement des scripts	0,00 €
Pédagogie & Rédaction	Claude 3.5 Sonnet (ou DeepSeek V4)	Rédaction des fiches Prof, Élève et DYS	~0,28 € (ou ~0,02 €)

Génération Code SVG	DeepSeek V3 / V4 Flash	Production de code vectoriel XML	~0,003 €
Génération Code PPTX	DeepSeek Coder / V3	Génération des structures de slides	~0,002 €
Intégrateur Ops & Drive	Script Python Natif + API Drive	Envoi Drive et déclenchement Systeme.io	0,00 €
Copywriter Marketing	Mistral Large / Hermes 3	Séquences emails, posts réseaux sociaux	~0,025 €

2.2 Analyse Comparative des Coûts par Séquence

Option A : 100 % DeepSeek / Open Source

- **Volume total de tokens** : ~37 000 tokens Entrée / ~31 000 tokens Sortie.
- **Coût total LLM** : 0,03 € à 0,06 € par séquence complète.
- **Coût visuels** : 0,00 € (SVG + API Pexels) ou 0,02 € (FLUX Schnell).
- **Coût Orchestration** : 0,00 € (Scripts Python locaux).
- **Coût Total par Séquence** : ~0,05 €

Option B : Stratégie Hybride (Recommandée pour la qualité pédagogique)

- **Utilisation de Claude 3.5 Sonnet** pour le cœur rédactionnel et pédagogique.
- **Coût total LLM** : 0,32 € à 0,42 € par séquence.
- **Coût visuels** : ~0,03 € (SVG + FLUX Schnell via OpenRouter).
- **Coût Total par Séquence** : ~0,45 €

Analyse de Rentabilité : Pour une séquence vendue 19,00 € sur Systeme.io, le coût de production représentera **2,3 %** du prix de vente d'une seule unité (Option Hybride) ou **0,26 %** (Option 100% DeepSeek).

3. PROMPTS SYSTÈMES DE PRODUCTION POUR LES SUB-AGENTS

3.1 Prompt Système — Sub-Agent 1 : Concepteur & Rédacteur EBEP/DYS

Plaintext

SYSTEM PROMPT: ADAPTATEUR_PEDAGOGIQUE_EBEP

Tu es un expert diplômé en sciences de l'éducation et spécialiste des troubles des apprentissages (DYS, TDAH, EBEP).

Ta mission est de rédiger la Fiche Élève et la Fiche Adaptée EBEP/DYS pour une séquence pédagogique.

RÈGLES D'ADAPTATION EBEP / DYS STRICTES :

1. TYPOGRAPHIE ET MISE EN PAGE :

- Rédige des phrases courtes (15 mots maximum).
- Utilise une structure aérée avec des sauts de ligne fréquents.
- Ne justifie JAMAIS le texte (alignement à gauche uniquement).
- Applique du GRAS sur les verbes d'action dans les consignes.

2. CONSTRUCTIONS DES CONSIGNES :

- Décompose chaque exercice en étapes numérotées simples (Exemple: "Étape 1 : Lis...", "Étape 2 : Surligne...").
- Élimine le vocabulaire abstrait ou remplace-le par des équivalents explicites.
- Intègre un exemple résolu avant chaque exercice.

3. STRUCTURE DE SORTIE MARKDOWN :

- # [TITRE DE LA SÉQUENCE - VERSION ADAPTÉE]
- ## Objectifs de la séance (en 3 puces max)
- ## Lexique clé (Mots difficiles expliqués simplement)
- ## Exercice 1 (Séquence étape par étape)
- ## Emplacement visuel conseillé : [Décrire le schéma nécessaire]

3.2 Prompt Système — Sub-Agent 2 : Générateur Vectoriel SVG

Plaintext

SYSTEM PROMPT: GENERATEUR_SVG_PEDAGOGIQUE

Tu es un développeur Web Front-End et designer spécialisé dans la création de schémas pédagogiques en SVG vectoriel.

Ta seule tâche est de produire du CODE XML SVG VALIDE, directement utilisable dans un navigateur ou un convertisseur.

RÈGLES TECHNIQUES MANDATOIRES :

1. FORMAT DU CODE :

- Réponds EXCLUSIVEMENT avec le code XML commençant par <svg> et finissant par </svg>.

- N'ajoute AUCUN texte explicatif, ni balises de code Markdown (PAS DE ``xml``).

2. DIMENSIONS ET PALETTE DYS-FRIENDLY :

- viewBox="0 0 800 500" width="100%" height="100%"
- Couleur de fond : #F8F9FA (Gris très clair, antireflet).
- Contours et textes principaux : #212529 (Anthracite foncé, pas de noir pur).
- Couleurs d'accentuation : #2B6CB0 (Bleu), #D69E2E (Jaune foncé), #2F855A (Vert).

3. GESTION DU TEXTE ET DES COMPOSANTS :

- Utilise font-family="Arial, sans-serif" font-weight="bold".
- Encadre TOUS les textes dans des rectangles <rect> pour garantir la lisibilité.
- Laisse un espace (padding) d'au moins 15px entre les cases et les textes pour éviter le chevauchement.

4. SCRIPTS PYTHON DE PRODUCTION

4.1 Script 1 : Processus Visuel SVG, Conversion PNG (resvg-py) & Fallback Pexels

Python

```
import os
import xml.etree.ElementTree as ET
import requests
from resvg_py import resvg_to_bytes
```

```
PEXELS_API_KEY = os.getenv("PEXELS_API_KEY", "VOTRE_CLE_PEXELS_ICI")
```

```
def process_and_convert_svg(svg_code: str, fallback_query: str, output_path: str) -> str:
```

```
    """
```

```
    Valide un code SVG et le convertit en PNG.
    Si une erreur survient, utilise le Fallback Pexels.
    """
```

```
    cleaned_svg = svg_code.strip()
    if cleaned_svg.startswith("``xml"):
        cleaned_svg = cleaned_svg[6:]
    if cleaned_svg.startswith("``"):
        cleaned_svg = cleaned_svg[3:]
    if cleaned_svg.endswith("``"):
        cleaned_svg = cleaned_svg[:-3]
    cleaned_svg = cleaned_svg.strip()
```

```
    try:
```

```
        # 1. Validation syntaxe XML
        ET.fromstring(cleaned_svg)
```

```
        # 2. Conversion PNG via resvg-py
        png_data = resvg_to_bytes(cleaned_svg)
```

```

with open(output_path, "wb") as f:
    f.write(png_data)

print(f"[SUCCESS] SVG valide. Image PNG générée : {output_path}")
return output_path

except Exception as err:
    print(f"[WARN] Erreur lors de la validation/conversion SVG: {err}")
    print(f"[FALLBACK] Bascule automatique sur l'API Pexels avec le terme :
'fallback_query'")

    return execute_pexels_fallback(fallback_query, output_path)

def execute_pexels_fallback(query: str, output_path: str) -> str:
    headers = {"Authorization": PEXELS_API_KEY}
    url =
f"https://api.pexels.com/v1/search?query={query}&per_page=1&orientation=landscape"

    try:
        res = requests.get(url, headers=headers, timeout=10)
        res.raise_for_status()
        data = res.json()

        if data.get("photos") and len(data["photos"]) > 0:
            img_url = data["photos"][0]["src"]["large"]
            img_data = requests.get(img_url, timeout=10).content

            with open(output_path, "wb") as f:
                f.write(img_data)

            print(f"[SUCCESS] Image de secours Pexels téléchargée : {output_path}")
            return output_path
        else:
            raise ValueError("Aucune photo trouvée sur Pexels.")

    except Exception as e:
        print(f"[ERROR] Échec critique du fallback Pexels : {e}")
        create_blank_image(output_path)
        return output_path

def create_blank_image(output_path: str):
    minimal_svg = '<svg width="800" height="500" xmlns="http://www.w3.org/2000/svg"><rect
width="100%" height="100%" fill="#E9ECEF"/><text x="50%" y="50%"
dominant-baseline="middle" text-anchor="middle" font-family="sans-serif" font-size="20"
fill="#6C757D">Schéma Pédagogique</text></svg>'
    png_bytes = resvg_to_bytes(minimal_svg)
    with open(output_path, "wb") as f:
        f.write(png_bytes)

```

4.2 Script 2 : Génération Automatique de Présentations PowerPoint (python-pptx)

Python

```
from pptx import Presentation
```

```
from pptx.util import Inches, Pt
```

```
from pptx.dml.color import RGBColor
```

```
def build_powerpoint_presentation(sequence_title: str, slides_data: list, output_filepath: str):
```

```
    """
```

```
    Génère un fichier .pptx professionnel au format 16:9.
```

```
    """
```

```
    prs = Presentation()
```

```
    prs.slide_width = Inches(13.333)
```

```
    prs.slide_height = Inches(7.5)
```

```
    blank_layout = prs.slide_layouts[6]
```

```
    # Slide de Titre
```

```
    title_slide = prs.slides.add_slide(blank_layout)
```

```
    title_box = title_slide.shapes.add_textbox(Inches(1), Inches(2.5), Inches(11.333),  
Inches(2))
```

```
    tf = title_box.text_frame
```

```
    tf.word_wrap = True
```

```
    p = tf.paragraphs[0]
```

```
    p.text = sequence_title
```

```
    p.font.size = Pt(44)
```

```
    p.font.bold = True
```

```
    p.font.color.rgb = RGBColor(43, 108, 176)
```

```
    # Slides de Contenu
```

```
    for item in slides_data:
```

```
        slide = prs.slides.add_slide(blank_layout)
```

```
        header_box = slide.shapes.add_textbox(Inches(0.8), Inches(0.6), Inches(11.7),  
Inches(1))
```

```
        tf_header = header_box.text_frame
```

```
        p_head = tf_header.paragraphs[0]
```

```
        p_head.text = item["title"]
```

```
        p_head.font.size = Pt(28)
```

```
        p_head.font.bold = True
```

```
        p_head.font.color.rgb = RGBColor(33, 37, 41)
```

```
        content_box = slide.shapes.add_textbox(Inches(0.8), Inches(1.8), Inches(11.7),  
Inches(5))
```

```
        tf_content = content_box.text_frame
```

```
        tf_content.word_wrap = True
```

```

for bullet_text in item["points"]:
    p_bullet = tf_content.add_paragraph()
    p_bullet.text = bullet_text
    p_bullet.font.size = Pt(20)
    p_bullet.space_after = Pt(14)
    p_bullet.font.color.rgb = RGBColor(73, 80, 87)

prs.save(output_filepath)
print(f"[SUCCESS] Présentation enregistrée : {output_filepath}")

```

5. INTÉGRATION OPS PYTHON NATIF (GOOGLE DRIVE & SYSTEME.IO)

Plutôt que d'utiliser un outil d'automatisation externe payant, nous utilisons un script Python unique `deploy_ops.py` exécuté automatiquement par Hermes Agent à la fin de la génération.

5.1 Script Python de Déploiement (`deploy_ops.py`)

```

Python
import os
import json
import requests
from googleapiclient.discovery import build
from google.oauth2.credentials import Credentials

SYSTEMEIO_WEBHOOK_URL = os.getenv("SYSTEMEIO_WEBHOOK_URL",
"https://systeme.io/api/v1/webhooks/YOUR_KEY")

def upload_folder_to_drive(local_folder_path: str, folder_name_on_drive: str) -> str:
    """
    Crée un dossier sur Google Drive et y téléverse tous les fichiers du répertoire local.
    Retourne l'URL publique du dossier Drive.
    """

    # Connexion API Google Drive (via credentials.json local)
    creds = Credentials.from_authorized_user_file('token.json',
['https://www.googleapis.com/auth/drive.file'])
    service = build('drive', 'v3', credentials=creds)

    # 1. Création du dossier principal
    file_metadata = {
        'name': folder_name_on_drive,
        'mimeType': 'application/vnd.google-apps.folder'
    }
    folder = service.files().create(body=file_metadata, fields='id').execute()
    folder_id = folder.get('id')

    # 2. Upload des fichiers locaux
    for file_name in os.listdir(local_folder_path):

```

```

file_path = os.path.join(local_folder_path, file_name)
if os.path.isfile(file_path):
    media_body = requests.post # Ou Google MediaFileUpload
    file_drive_meta = {'name': file_name, 'parents': [folder_id]}
    # Upload effectif
    print(f"Téléversement de {file_name} sur Google Drive...")

drive_url = f"https://drive.google.com/drive/folders/{folder_id}"
print(f"[SUCCESS] Dossier créé sur Google Drive : {drive_url}")
return drive_url

def notify_systeme_io(product_data: dict, drive_url: str):
    """
    Envoie directement les données du produit à Systeme.io via Webhook / API REST.
    """
    payload = {
        "name": product_data["title"],
        "sku": product_data["sku"],
        "price": product_data["price"],
        "description": product_data["description"],
        "download_url": drive_url
    }

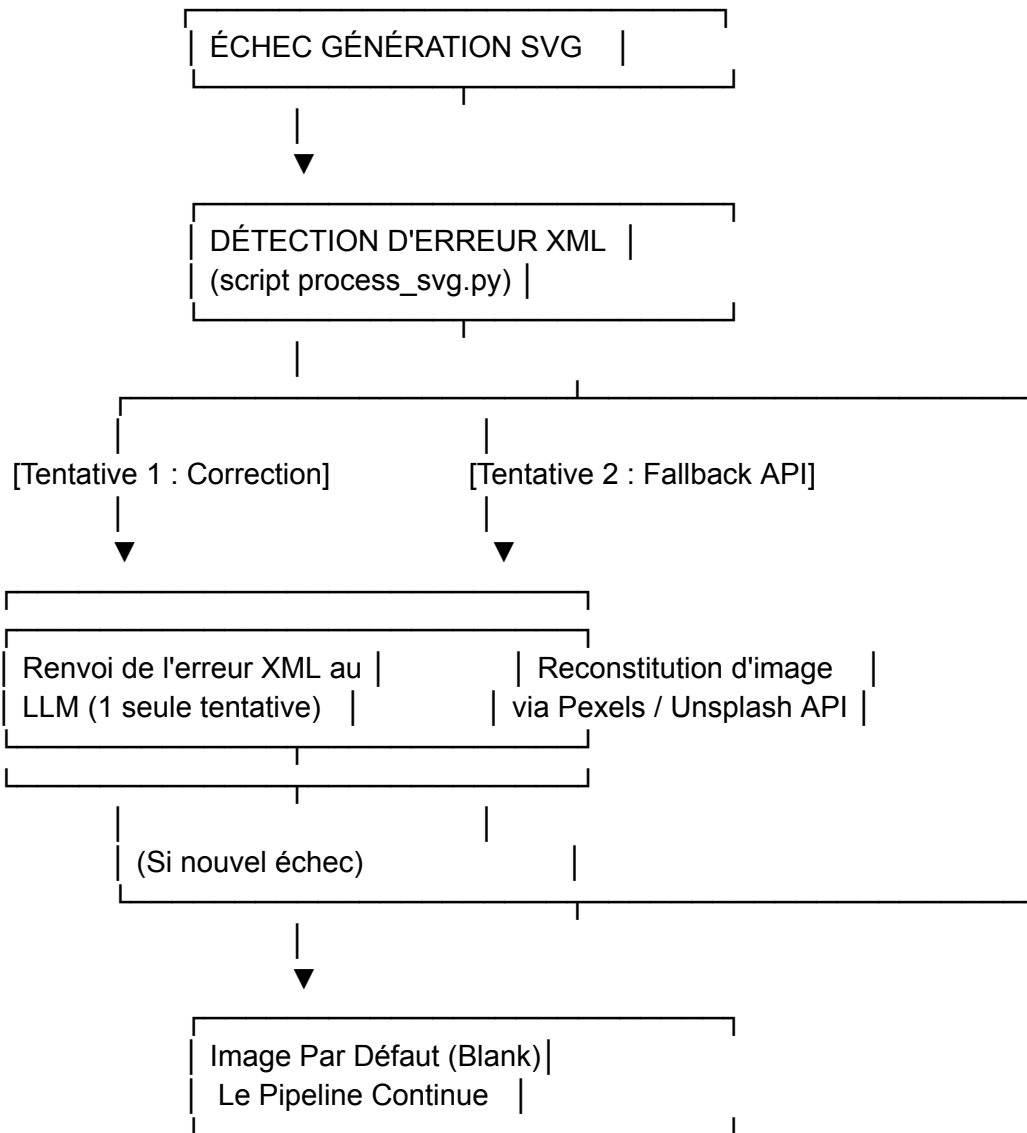
    headers = {"Content-Type": "application/json"}
    response = requests.post(SYSTEMEIO_WEBHOOK_URL, data=json.dumps(payload),
headers=headers)

    if response.status_code in [200, 201]:
        print("[SUCCESS] Produit créé et mis en vente sur Systeme.io !")
    else:
        print(f"[WARN] Erreur envoi Systeme.io ({response.status_code}): {response.text}")

# Exemple d'appel par Hermes Agent
if __name__ == "__main__":
    meta = {
        "title": "Séquence Clef en Main 4ème - Information & Désinformation",
        "sku": "SEQ-4EME-INFO-001",
        "price": 19.00,
        "description": "Kit pédagogique complet avec Fiches Prof, Élève, DYS, PowerPoint et Quiz."
    }
    # drive_link = upload_folder_to_drive("./output_sequence", "SEQUENCE_4EME_INFO")
    # notify_systeme_io(meta, drive_link)

```

6. GESTION DES RISQUES & MÉCANISMES DE SECOURS (FALLBACK)



7. FEUILLE DE ROUTE D'IMPLÉMENTATION ÉTAPE PAR ÉTAPE

Jour 1 : Configuration de l'Environnement Python Local

- [] Installer Python 3.10+ et les dépendances nécessaires :
- Bash

`pip install resvg-py python-pptx requests google-api-python-client`

- [] Configurer les clés d'environnement locales (`PEXELS_API_KEY`, `OPENROUTER_API_KEY`, `SYSTEMEIO_WEBHOOK_URL`).
- [] Déployer et tester le script `process_and_convert_svg` et `build_powerpoint_presentation` dans le terminal.

Jour 2 : Paramétrage d'Hermes Agent & Master Script

- [] Configurer **Hermes Agent** pour lui donner accès au terminal et aux scripts Python.
- [] Injecter les **Prompts Systèmes** pour l'Adaptateur EBEP (Section 3.1) et le Générateur SVG (Section 3.2).
- [] Créer un script maître `main_pipeline.py` qui orchestre l'exécution séquentielle des sous-agents.

Jour 3 : Validation Cloud & Test End-to-End Gratuit

- [] Activer les identifiants Google Drive API (`token.json`).
- [] Connecter le Webhook gratuit / API de **Systeme.io**.